

Research on Multi-Fidelity Data-Based Optimization Algorithms

Wenjie Ma

2026-10-05

Beijing Institute of Technology

What is multi-fidelity?

- **Fidelity:** how *faithful* a test is
- **High-fidelity:** *expensive*, but precise
- **Low-fidelity:** *cheap*, but a little off
- **Multi-fidelity:** use *both*

How to use them together to find the best answer?

Here's my plan

1. A quick story
2. Two simple ideas
3. What I found
4. Where I'd like to go next

A quick story

Testing is expensive, few chances

You work for a company, but —

- Tune parameters of an *experiment* — every run costs money
- More than 10 parameters, and they *affect each other*
- Can't tune them *one at a time* — need the best combination
- Engines, materials, biology: the same story

How to spend a tiny budget smartly?

Why is it so hard? Two reasons

Two reasons:

- **Black box:** only input and output, never the inside — a simulation or test bench
- **Budget:** an enormous parameter space, but only about 15 runs

Use 15 runs to find the best point of a high-dimensional black box.

Good news: two fidelities

Two fidelities:

- **Low-fidelity:** cheap & fast, but *biased*
- **High-fidelity:** accurate, but only 15 runs

How do we use the cheap low-fidelity runs to guide the expensive high-fidelity ones?

My simple idea

My idea is simple:

$$h_k(\mathbf{x}) = h_{k-1}(\mathbf{x}) + \delta_k(\mathbf{x})$$

Learn the correction δ_k from a few high-fidelity runs

Two simple ideas

Where to put the first runs?

— **maxpro design**

How to add runs one at a time?

— **sequential design**



V. Roshan Joseph

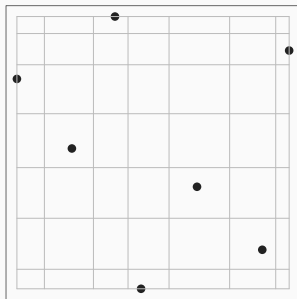
First piece: maxpro design

Maxpro:

maximum projection. spread out in the full space, and in *every low-dimensional shadow*.

Criterion:

$$\min_D \sum_{i=1}^n \prod_{j \neq i} \frac{1}{\sum_{k=1}^p (x_{ik} - x_{jk})^2}$$



The MaxPro Design

Second piece: Gaussian progress

- A **normal distribution** describes *one number*: the average, and how spread out
- A **Gaussian process** is the same idea, but for a *whole function*: a whole curve, plus *how unsure* we are at each point
- That *uncertainty* is the guide — test where we are *hopeful and uncertain*

The whole pipeline

1. Choose **starting points** — spread out evenly
2. Feed **many cheap low-fidelity points**; the GP draws a *rough map*
3. Run **one expensive high-fidelity test** where the map says the *destination*
4. **Repeat** — the map gets sharper every round

Map, guess, test, refine.

The formal question

The formal question:

$$\hat{\mathbf{x}} \approx \arg \min_{\mathbf{x} \in [0,1]^p} h_{\text{H}}(\mathbf{x})$$

Find the best point using only **15 high-fidelity tests**.

Three smaller questions:

1. Does **combining the two fidelities** help?
2. Does **screening** help — which parameters matter, drop the rest?
3. Does it still work in **high dimensions**?

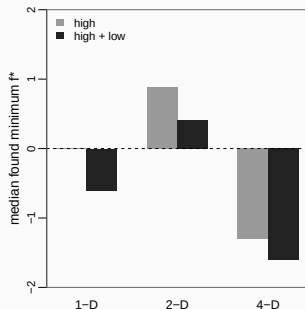
How I measure success

- Run the pipeline **20 times** from scratch
- Take the **median** of the 20 best values
- **Lower is better** in every figure

What I found

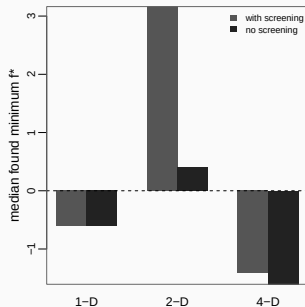
Result 1: combining fidelities is the key

Don't just run the expensive one — **bring a cheap one along to help.**



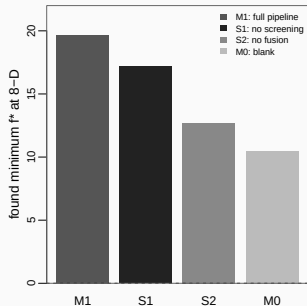
Result 2: screening is useless

Screening sounds like it saves work, but it **costs you accuracy**

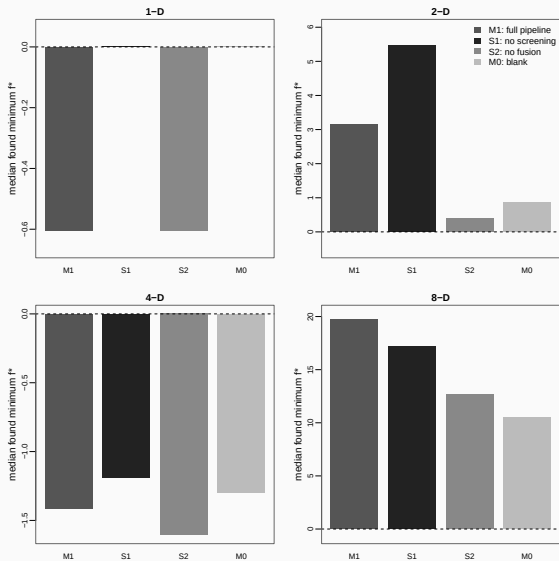


Result 3: high dimensions fail

In eight dimensions, **high dimensions fail** — the curse of dimensionality.



All results in one picture



Where I'd like to go next

Three directions next

1. When is screening trustworthy?
2. Break the curse of dimensionality
3. Push it into the real world

Summary

Don't just **run more high-fidelity tests** — **make every single one count**. Let cheap low-fidelity runs do the **heavy lifting**.

That's **multi-fidelity**.

Thank you for your listening!